



Docs Menu



The Cloud Resume Challenge - GCP

> NEW: Check out the new 2025 edition of the Cloud Resume Challenge Guidebook for Azure!

1. Certification

Your resume needs to have the Google Cloud Digital Leader certification on it. This is an introductory certification that orients you on Google Cloud – if you have a more advanced Google Cloud cert, that's fine but not expected. You can sit this exam online for $100 USD.

NEW: ExamPro, a Cloud Resume Challenge sponsor, offers a full Google Cloud Digital Leader (GCP-CDL) course with both **free** and **paid** study options. You can access free video lessons and a practice exam without needing a credit card — giving you everything you need to prepare and pass the exam.

2. HTML

Your resume needs to be written in HTML. Not a Word doc, not a PDF. Here is an example of what I mean.

3. CSS

Your resume needs to be styled with CSS. No worries if you're not a designer – neither am I. It doesn't have to be fancy. But we need to see something other than raw HTML when we open the webpage.

4. Static Website

Your HTML resume should be deployed online as a static website using Google Cloud Storage. Services like Netlify and GitHub Pages are great and I would normally recommend them for personal static site deployments, but they make things a little too abstract for our purposes here. Use GCS.

5. HTTPS

The GCS website URL should use HTTPS for security. You will need to use a Cloud Load Balancer to help with this. Be aware - this will have a small cost to provision. You will also want to use Cloud CDN to cache your site - just check the box when configuring your load balancer backend.

6. DNS

Point a custom DNS domain name to the load balancer endpoint, so your resume can be accessed at something like `my-c00l-resume-website.com`. You can use Cloud Domains or any other DNS provider for this. A domain name usually costs about ten bucks to register.

7. Javascript

Your resume webpage should include a visitor counter that displays how many people have accessed the site. You will need to write a bit of Javascript to make this happen. Here is a helpful tutorial to get you started in the right direction.

8. Database

The visitor counter will need to retrieve and update its count in a database somewhere. I suggest you use Google Cloud's Firestore for this. (It might be a bit

overkill to use Distributed Counters, but it could also be fun!) Be aware that Firestore may incur small costs to store and manipulate data.

9. API

Do not communicate directly with Firestore from your Javascript code. Instead, you will need to create an API that accepts requests from your web app and communicates with the database. I suggest using Google Cloud Functions for this with an HTTP trigger. They will be free or close to free for what we are doing.

10. Python

You will need to write a bit of code in the serverless function; you could use more Javascript, but it would be better for our purposes to explore Python – a common language used in back-end programs and scripts – and its Google Cloud Client Libraries. Here is a good, free Python tutorial.

11. Tests

You should also include some tests for your Python code. Here are some resources on writing good Python tests.

12. Infrastructure as Code

You should not be configuring your API resources – the Firestore, the API Gateway, the serverless function – manually, by clicking around in the Google Cloud console. Instead, define them in a Terraform template and deploy them using the Terraform CLI. This is called "infrastructure as code" or IaC. It saves you time in the long run.

13. Source Control

You do not want to be updating either your back-end API or your front-end website by making calls from your laptop, though. You want them to update automatically whenever you make a change to the code. (This is called continuous integration and deployment, or CI/CD.) Create a GitHub repository for your backend code.

14. CI/CD (Back end)

Set up Cloud Build such that when you push an update to your Terraform config or Python code, your Python tests get run. If the tests pass, the application should get packaged and deployed to Google Cloud.

15. CI/CD (Front end)

Create a second GitHub repository for your website code. Create a Cloud Build configuration such that when you push new website code, the Cloud Storage bucket automatically gets updated. (You may need to invalidate your Cloud CDN cache in the code as well.) *Important note: DO NOT commit Google Cloud credentials to source control! Bad hats will find them and use them against you!*

16. Blog post

Finally, in the text of your resume, you should link a short blog post describing some things you learned while working on this project. Dev.to or Hashnode are great places to publish if you don't have your own blog.

And that's the gist of it! For strategies, tools, and further challenges to help you get hired in cloud, check out the GCP edition of the Cloud Resume Challenge book.

The Cloud Resume ... The Cloud Resume ...

© 2026 ExamPro Training Inc.